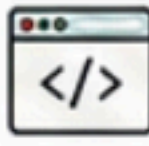


Q.1

EXPLAIN DIFFERENT LAYERS OF IoT COMMUNICATION PROTOCOLS

IoT communication uses a layered architecture similar to the OSI model. Each layer has specific functions to ensure reliable data transfer between devices and the cloud.

LAYER	FUNCTION	EXAMPLES / PROTOCOLS
5. Application Layer	- Provides services to user applications. - Defines how applications interact with IoT devices. 	- HTTP, MQTT, CoAP, AMQP, WebSocket - REST API
4. Transport Layer	- Ensures end-to-end data delivery, segmentation and reliability. - Handles error recovery and flow control. 	- TCP (Transmission Control Protocol) - UDP (User Datagram Protocol)
3. Network Layer	- Handles logical addressing and routing of data. - Selects best path for data delivery. 	- IP (IPv4, IPv6) - ICMP, RPL (for Low power IoT)
2. Data Link Layer	- Provides node-to-node data transfer. - Framing, error detection and correction using MAC address. 	Ethernet, Wi-Fi (802.11), Bluetooth, ZigBee, LoRaWAN (MAC)
1. Physical Layer	- Transmits raw bits over physical medium. - Defines electrical, mechanical and radio specifications. 	802.11 PHY, BLE PHY, LoRa, GSM, NB-IoT

Note : Lower layers handle data transmission, upper layers handle application services.

Q.2

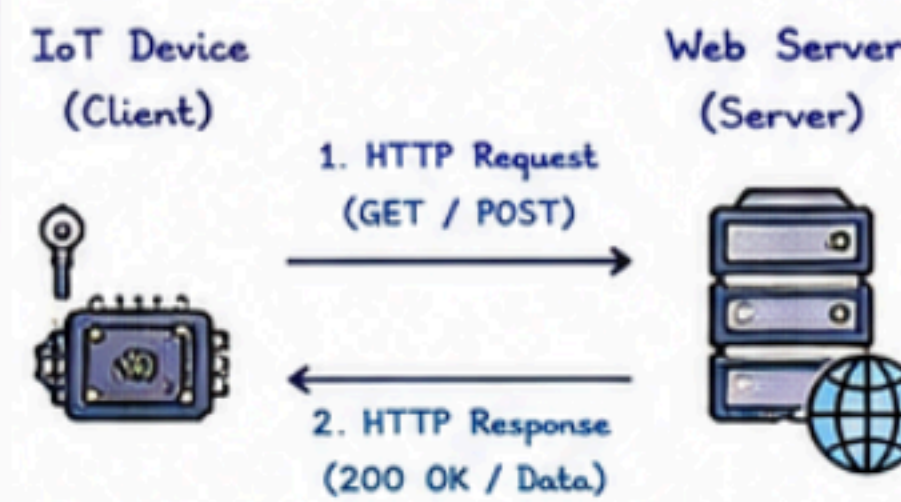
WHAT IS HTTP ? EXPLAIN ITS ROLE IN IoT APPLICATION

HTTP (HyperText Transfer Protocol) is an application layer protocol used for transferring data over the web. It is the foundation of data communication in IoT based web services.

KEY FEATURES

- Application layer protocol.
- Works on request-response model.
- Uses TCP as transport protocol.
- Stateless protocol.
- Uses URLs to identify resources.
- Common methods: GET, POST, PUT, DELETE.

WORKING (HTTP COMMUNICATION)



ROLE IN IoT APPLICATION

- Data Transfer :** IoT devices send sensor data to web servers using HTTP.
- Integration :** Easily integrates with cloud platforms and web services (REST APIs).
- Interoperability :** Works with any system that supports HTTP.
- Monitoring & Control :** IoT devices can be monitored and controlled through web dashboards or mobile apps.
- Simple & Widely Supported :** Supported by browsers, servers and many IoT platforms.

EXAMPLE

An IoT temperature sensor sends data to the cloud using HTTP POST request and the server stores it in a database. The user can view the data on a web dashboard.

Q.3

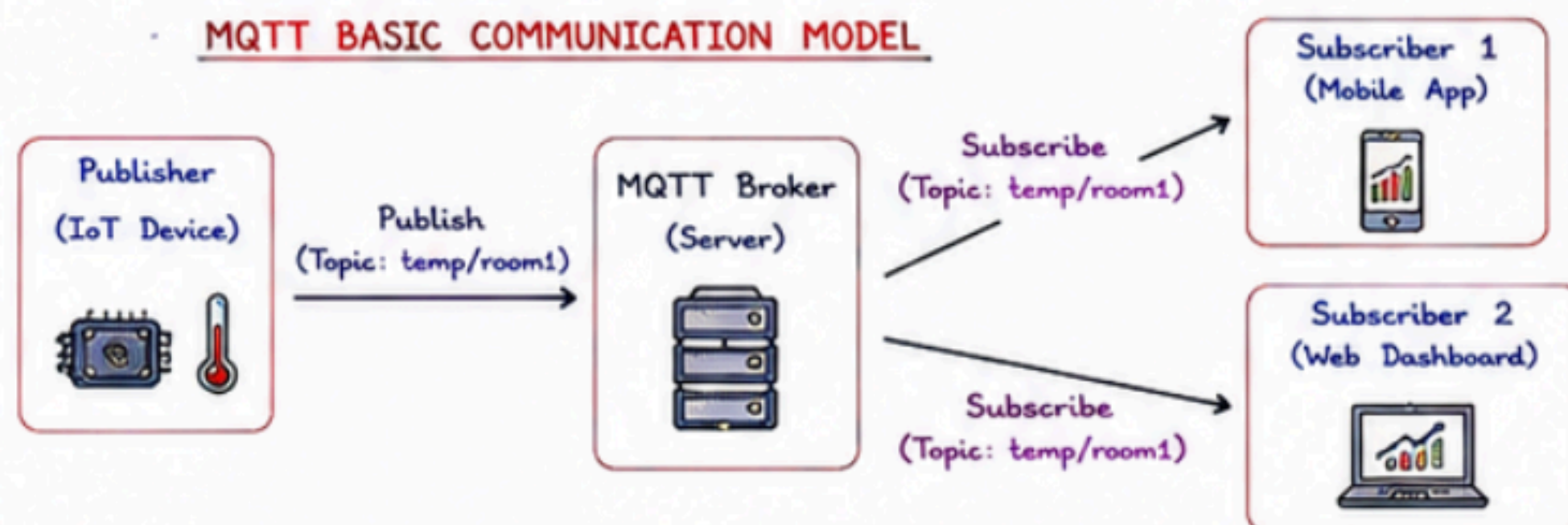
EXPLAIN MQTT AND ITS BASIC COMMUNICATION MODEL

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed for IoT. It runs over TCP/IP and is optimized for low bandwidth and unreliable networks.

KEY FEATURES

- Lightweight and simple.
- Uses publish-subscribe model.
- Low overhead, ideal for constrained devices.
- Works over TCP/IP.
- Supports Quality of Service (QoS 0, 1, 2).
- Retained messages and Last Will (for reliability).

MQTT BASIC COMMUNICATION MODEL



HOW IT WORKS

- Publisher (IoT device) publishes messages to a specific topic on the broker.
- Broker receives the message and forwards it to all subscribers of that topic.
- Subscribers receive the message and act on the data.

EXAMPLE

- A temperature sensor (publisher) publishes data to topic "home/temperature".
- Mobile app and web dashboard (subscribers) subscribe to the same topic.
- Broker sends the temperature data to all subscribers in real time.

MQTT QoS LEVELS

QoS 0 (At most once): Message may be lost, no acknowledgement.

QoS 1 (At least once): Message is delivered at least once.

QoS 2 (Exactly once): Message is delivered exactly once.

APPLICATIONS : Smart home, Industrial monitoring, Remote monitoring, Asset tracking, Smart agriculture, etc.

Q.4 DIFFERENCE BETWEEN CoAP AND MQTT

CoAP (Constrained Application Protocol) and MQTT (Message Queuing Telemetry Transport) are both IoT protocols but they differ in design, communication model and usage.

Basis	CoAP	MQTT
1. Full Form	Constrained Application Protocol	Message Queuing Telemetry Transport
2. Type	Application layer protocol	Application layer messaging protocol
3. Transport	Runs over UDP	Runs over TCP
4. Communication Model	RESTful (Request - Response) Client ↔ Server	Publish - Subscribe Client ↔ Broker ↔ Clients
5. Message Format	Binary, compact header (4 bytes) Uses options for extensibility	Fixed header (2 bytes), variable header and payload
6. Reliability	Non-reliable over UDP; Confirmable messages provide reliability	Reliable over TCP
7. Overhead	Very low (suitable for constrained devices) Small packet size	Higher than CoAP (due to TCP and larger header)
8. QoS Support	Confirmable / Non-confirmable messages	Three QoS levels (0, 1, 2)
9. Security	DTLS for security	TLS/SSL for security
10. Discovery	Resource discovery using .well-known/core	No built-in discovery (external mechanism needed)
11. Best Suited For	Constrained devices, low power networks, REST based interaction	Telemetry data, real-time messaging, when broker based pub/sub is required
12. Examples	Smart home devices, sensor nodes, industrial monitoring	IoT telemetry, remote monitoring, chat apps, M2M communication

Q.5 WRITE SHORT NOTE ON WEBSOCKET, XMPP, DDS AND AMQP

1. WebSocket

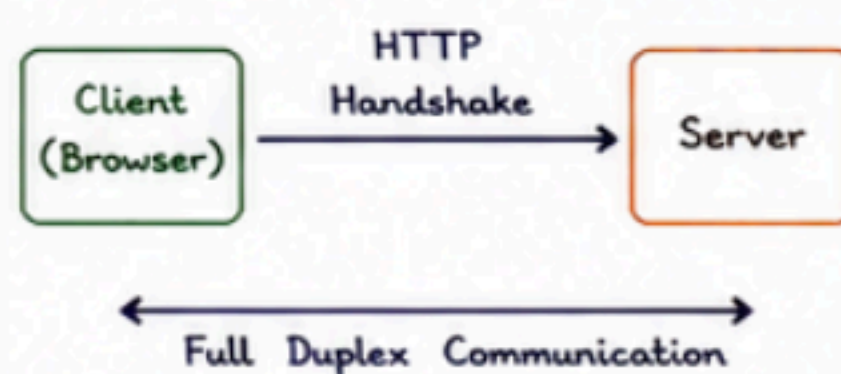
WebSocket is a communication protocol providing full-duplex bi-directional communication over a single TCP connection.

Key Points :

- Works over TCP.
- Starts with HTTP handshake (ws:// or wss://).
- After handshake, a persistent connection is established.
- Low overhead, suitable for real-time applications.

Applications :

- Real-time dashboards
- Online gaming
- Live chat, streaming data



2. XMPP

XMPP (Extensible Messaging and Presence Protocol) is an open standard protocol for real-time messaging and presence information.

Key Points :

- XML based protocol.
- Works over TCP.
- Uses client-server architecture.
- Supports messaging, presence, file transfer, voice/video and group chat.
- Extensible using extensions (XEPs).

Applications :

- Instant messaging
- Presence and status
- IoT device communication



3. DDS

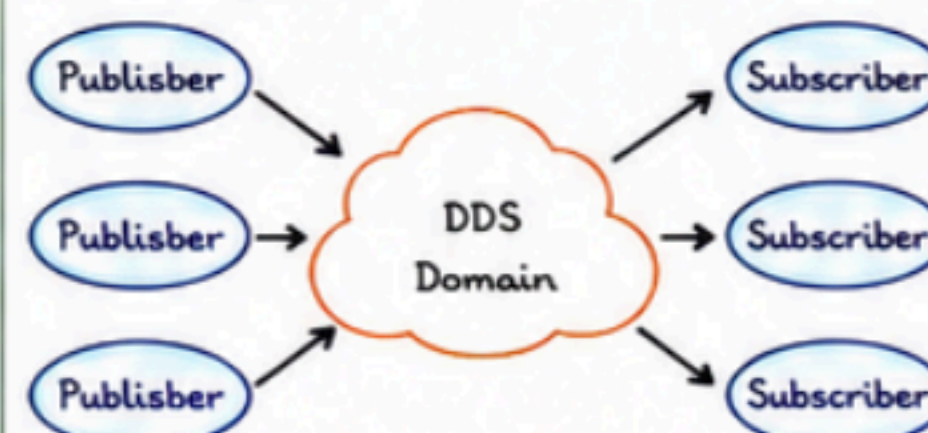
DDS (Data Distribution Service) is a data-centric publish-subscribe middleware protocol for real-time systems.

Key Points :

- Data-centric (focus on data not devices).
- Built-in publish-subscribe model.
- High performance, low latency.
- Automatic discovery, reliability and QoS management.
- Peer-to-peer or brokerless communication.

Applications :

- Industrial automation
- Autonomous vehicles
- Aerospace and defense



4. AMQP

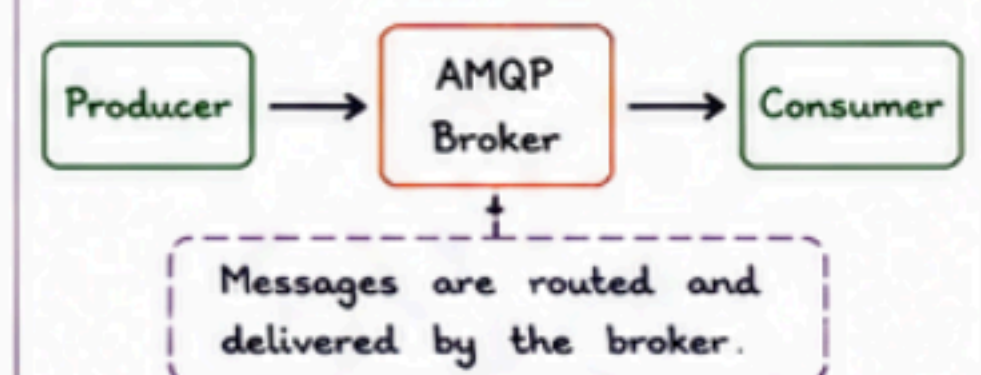
AMQP (Advanced Message Queuing Protocol) is an open standard application layer protocol for message-oriented middleware.

Key Points :

- Works over TCP.
- Provides reliable messaging between systems.
- Supports queues, routing, acknowledgements.
- Platform and language independent.

Applications :

- Enterprise messaging
- Queue management
- Financial systems



Q.6 Explain WebSocket in IoT

Definition:

"WebSocket is a communication protocol that provides full-duplex, two-way communication between an IoT device and a server over a single, persistent TCP connection."

Working:

1. The IoT device (client) sends a connection request to the server.
2. A WebSocket handshake is performed and the connection is established.
3. After connection, both client and server can send data at any time.
4. The connection remains open until it is closed by either side.

Features:

- Full-duplex communication
- Real-time data transfer
- Low communication overhead
- Persistent TCP connection

Example:

In a smart home, WebSocket can send sensor data to a mobile application in real time. The user can also send an ON/OFF command to control a light or fan.

Diagram:



Conclusion:

Thus, WebSocket is useful for real-time monitoring and control in IoT applications.

Q.7

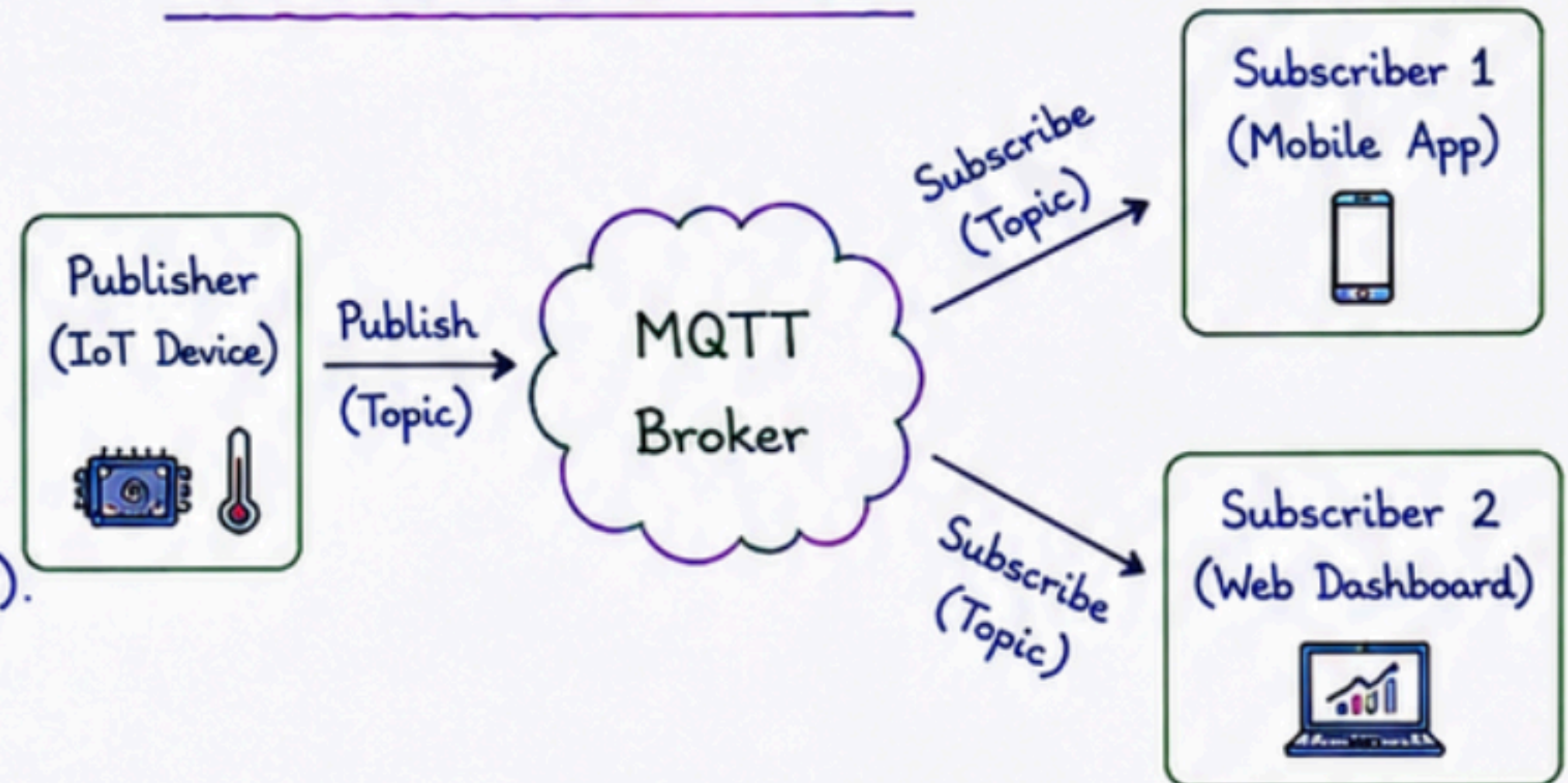
Explain MQTT in detail

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed for IoT applications. It works over TCP/IP and is optimized for low bandwidth, low power and unreliable networks.

1. Features of MQTT :

- Lightweight and simple.
- Uses publish-subscribe model.
- Works over TCP/IP.
- Low overhead, ideal for constrained devices.
- Supports Quality of Service (QoS 0, 1, 2).
- Retained messages and Last Will (for reliability).
- Secure communication using TLS/SSL.

2. MQTT Architecture :



3. How MQTT Works :

1. A publisher (IoT device) publishes a message to a topic on the broker.
2. The broker receives the message and forwards it to all subscribers who are subscribed to that topic.
3. Subscribers receive the message and act on the data.

4. Quality of Service (QoS) Levels :

- QoS 0 (At most once) : Message may be lost, no acknowledgement.
- QoS 1 (At least once) : Message is delivered at least once.
- QoS 2 (Exactly once) : Message is delivered exactly once.

5. MQTT Control Packets :

- CONNECT - Establish connection with broker.
- PUBLISH - Publish message to a topic.
- SUBSCRIBE - Subscribe to a topic.
- UNSUBSCRIBE - Unsubscribe from a topic.
- PINGREQ / PINGRESP - Keep alive messages.
- DISCONNECT - Close the connection.

6. Advantages :

- Efficient for low bandwidth and low power devices.
- Supports large number of clients.
- Provides real-time data transfer.
- Reliable with QoS and Last Will.
- Widely used in IoT applications.

7. Applications of MQTT :

- Smart home automation
- Industrial monitoring and control
- Remote monitoring of sensors
- Smart agriculture
- Asset tracking and telematics

Example :

A temperature sensor (publisher) publishes temperature data to topic "home/temperature". Mobile app and web dashboard (subscribers) subscribe to the same topic and receive the data in real time.

Conclusion :

Thus, MQTT is a lightweight and efficient protocol ideal for IoT applications that require reliable, real-time messaging with low resource consumption.

Q.8 Explain the major Application Layer Protocols used in IoT: HTTP, WebSocket, MQTT, CoAP, XMPP, DDS and AMQP. Compare their important characteristics and applications.

Answer :

Introduction : Application layer protocols in IoT provide services for communication between IoT devices and applications. Major protocols include HTTP, WebSocket, MQTT, CoAP, XMPP, DDS and AMQP.

Major Application Layer Protocols :

Protocol	Key Characteristics	Applications
HTTP	• Request-response protocol • Stateless communication • Runs over TCP/IP • Simple and widely used	• Web and IoT applications • Configuration and data access • Smart home, smart devices
WebSocket	• Full-duplex, two-way communication • Persistent TCP connection • Real-time data transfer	• Real-time monitoring and control • Live dashboards • Smart home, industrial IoT
MQTT	• Publish-subscribe model • Lightweight and low power • Reliable (QoS 0, 1, 2) • Runs over TCP/IP	• Sensor data transmission • Home automation • Connected devices
CoAP	• Constrained Application Protocol • Works over UDP • Low power and lightweight • RESTful model	• IoT for constrained devices • Smart meters • Wearables
XMPP	• XML based protocol • Publish-subscribe model • Supports presence and messaging	• IoT device communication • Smart grid • Presence and control
DDS	• Data Distribution Service • Real-time, high performance • Peer-to-peer communication • Low latency	• Industrial IoT • Autonomous systems • Real-time control
AMQP	• Advanced Message Queuing • Reliable message delivery • Works over TCP • Queuing and routing	• Enterprise IoT • Remote monitoring • Reliable data exchange

Comparison / Key Points :

- HTTP and WebSocket are web-oriented protocols (request-response and real-time).
- MQTT and CoAP are lightweight, publish-subscribe protocols suitable for constrained IoT devices.
- XMPP, DDS and AMQP are advanced protocols for messaging, real-time data distribution and reliable enterprise communication.

Conclusion : Each protocol is suited to different IoT needs — HTTP for web access, MQTT/CoAP for lightweight device communication, XMPP for messaging, DDS for real-time data and AMQP for reliable enterprise IoT systems.